Experiment 6: Gradient Descent, SGD, Momentum and Adam Comparison

Aim

To implement Batch Gradient Descent, Stochastic Gradient Descent (SGD), and Momentum from scratch, compare their optimization paths, train a Keras regression model using SGD, Momentum, and Adam, and investigate the effect of learning rate on convergence.

Formulas

1. Gradient Descent

$$\theta(new) = \theta(old) - \eta \times \nabla J(\theta)$$

2. Stochastic Gradient Descent (SGD)

$$\theta(new) = \theta(old) - \eta \times \nabla J_i(\theta)$$

3. Momentum

$$v(t) = \beta \times v(t-1) + \nabla J(\theta)$$

$$\theta(new) = \theta(old) - \eta \times v(t)$$

4. Adam Optimizer

$$m(t) = \beta_1 \times m(t-1) + (1 - \beta_1) \times g(t)$$

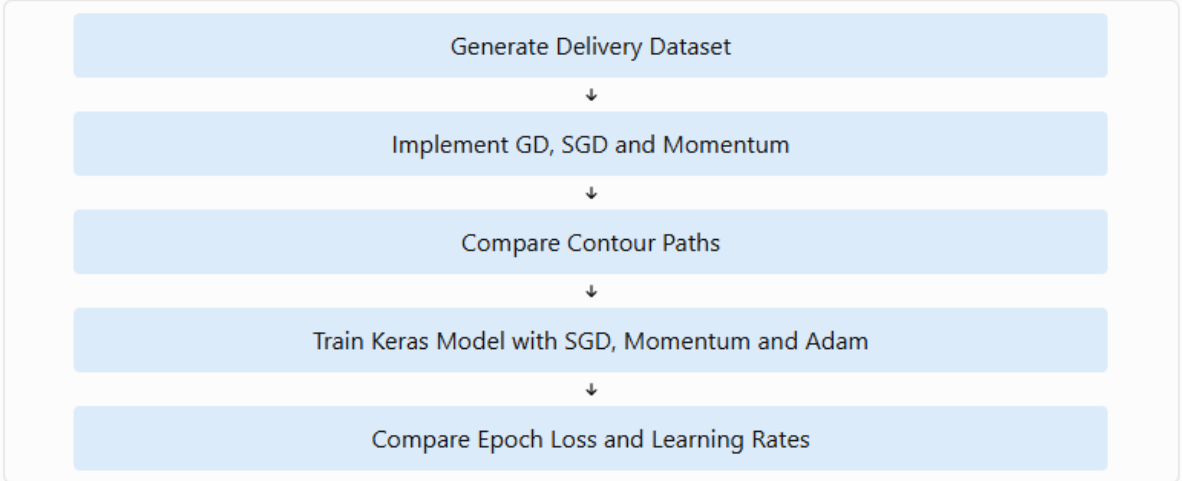
$$v(t) = \beta_2 \times v(t-1) + (1 - \beta_2) \times g(t)^2$$

$$\theta(new) = \theta(old) - \eta \times \hat{m}(t) / (\sqrt{\hat{v}(t)} + \varepsilon)$$

5. Mean Squared Error (MSE)

$$MSE = \Sigma(y_i - \hat{y}_i)^2 / N$$

Architecture diagram



Code

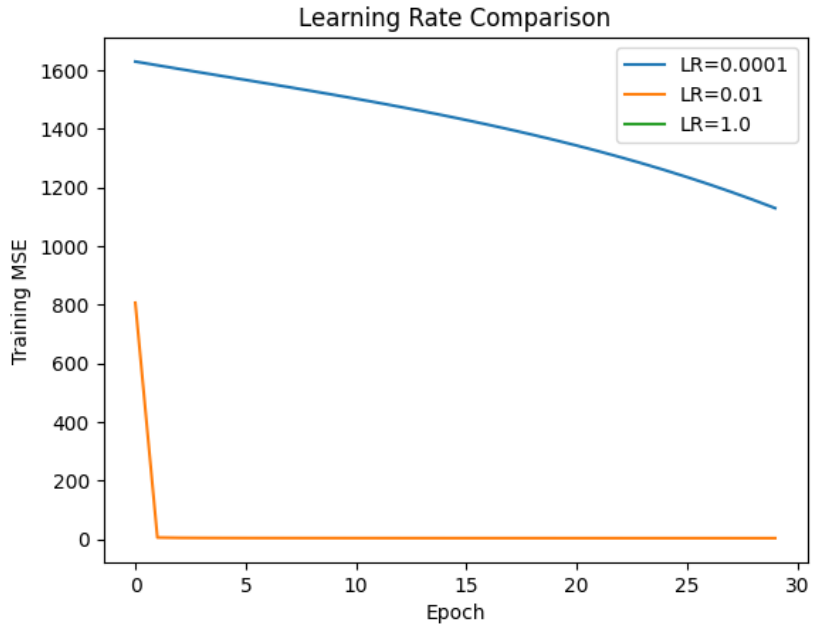
```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
np.random.seed(42)
tf.random.set_seed(42)
def loss_fn(w):
    return w[0]**2 + 5*w[1]**2
def gradient(w):
    return np.array([2*w[0], 10*w[1]])
def optimize(method, lr=0.1, steps=60):
    w = np.array([3.0, 2.0])
    velocity = np.zeros(2)
    path = [w.copy()]
    for _ in range(steps):
        if method == "SGD":
            g = np.zeros(2)
            j = np.random.randint(2)
            g[j] = gradient(w)[j] * 2
        else:
            g = gradient(w)
        if method == "Momentum":
            velocity = 0.9 * velocity + g
            w = w - lr * velocity
        else:
            w = w - lr * g
        path.append(w.copy())
    return np.array(path)
paths = {
```

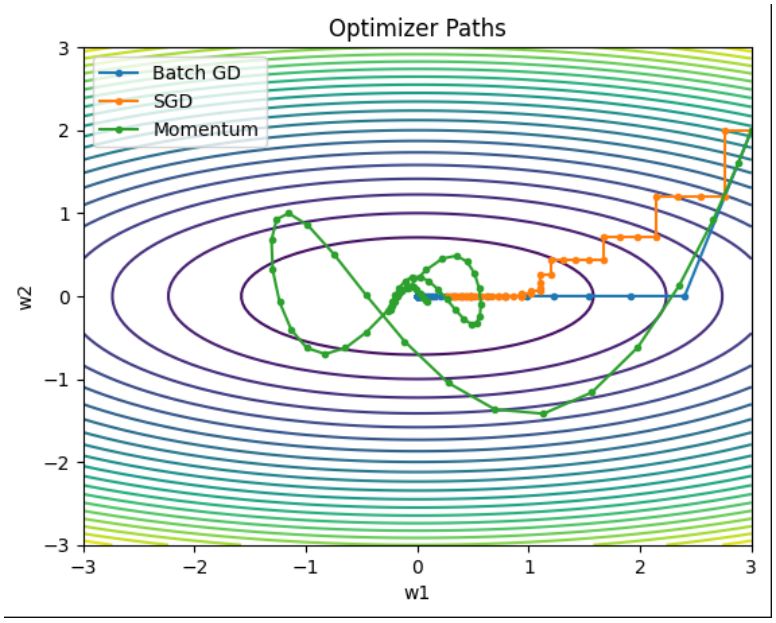
```
    "Batch GD": optimize("GD", 0.1),
    "SGD": optimize("SGD", 0.02),
    "Momentum": optimize("Momentum", 0.02)
}
a = np.linspace(-3, 3, 100)
b = np.linspace(-3, 3, 100)
A, B = np.meshgrid(a, b)
Z = A**2 + 5*B**2
plt.contour(A, B, Z, levels=25)
for name, path in paths.items():
    plt.plot(path[:, 0], path[:, 1], marker=".", label=name)
plt.xlabel("w1")
plt.ylabel("w2")
plt.title("Optimizer Paths")
plt.legend()
plt.show()
n = 300
X = np.random.rand(n, 4)
y = (10 + 30*X[:, 0] + 15*X[:, 1]
     + 5*X[:, 2] + 8*X[:, 3]
     + np.random.normal(0, 2, n))
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
def make_model(optimizer):
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(4,)),
        tf.keras.layers.Dense(16, activation="relu"),
```

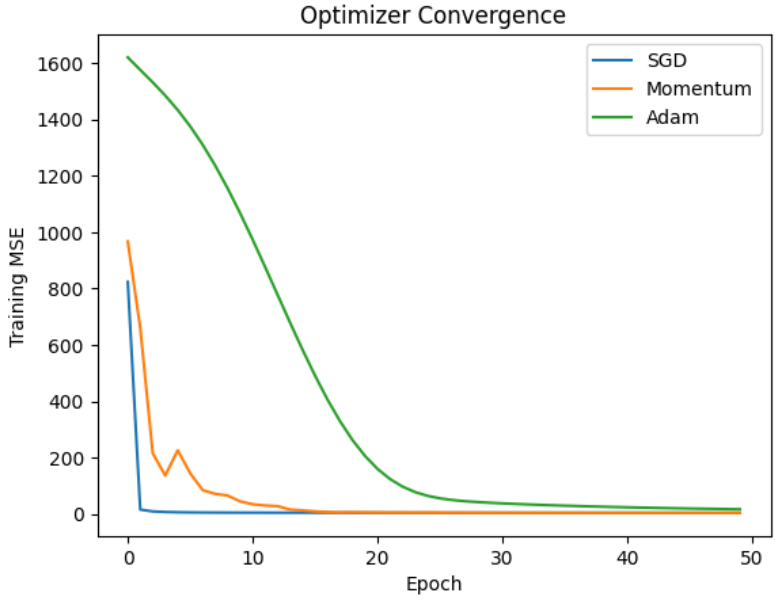
```
        tf.keras.layers.Dense(1)
    ])
    model.compile(optimizer=optimizer, loss="mse")
    return model
optimizers = {
    "SGD": tf.keras.optimizers.SGD(learning_rate=0.01),
    "Momentum": tf.keras.optimizers.SGD(
        learning_rate=0.01, momentum=0.9
    ),
    "Adam": tf.keras.optimizers.Adam(learning_rate=0.01)
}
histories = {}
for name, opt in optimizers.items():
    tf.keras.backend.clear_session()
    tf.random.set_seed(42)
    model = make_model(opt)
    histories[name] = model.fit(
        X_train, y_train,
        epochs=50, batch_size=32,
        validation_split=0.2, verbose=0
    ).history
for name, h in histories.items():
    plt.plot(h["loss"], label=name)
plt.xlabel("Epoch")
plt.ylabel("Training MSE")
plt.title("Optimizer Convergence")
plt.legend()
plt.show()
for lr in [0.0001, 0.01, 1.0]:
    tf.keras.backend.clear_session()
    tf.random.set_seed(42)
```

```
    model = make_model(tf.keras.optimizers.SGD(learning_rate=lr))
    h = model.fit(
        X_train, y_train, epochs=30,
        batch_size=32, verbose=0
    ).history["loss"]
    plt.plot(h, label=f"LR={lr}")
plt.xlabel("Epoch")
plt.ylabel("Training MSE")
plt.title("Learning Rate Comparison")
plt.legend()
plt.show()
print("Test MSE for each optimizer:")
for name, opt in optimizers.items():
    tf.keras.backend.clear_session()
    tf.random.set_seed(42)
    model = make_model(opt)
    model.fit(X_train, y_train, epochs=50, verbose=0)
    print(name, round(model.evaluate(X_test, y_test, verbose=0), 3))
```

output








```
Test MSE for each optimizer:
SGD 7.056
Momentum 6.493
Adam 19.422
```

Experiment 7: Perceptron and Forward Propagation

Aim

To implement a single perceptron using NumPy, train it using the perceptron learning rule, visualize its decision boundary, demonstrate its inability to learn XOR, and compare step, sigmoid and ReLU activations.

Formulas

1. Weighted Sum

$z = (w_1x_1 + w_2x_2 + \dots + w_nx_n) + b$

2. Step Activation Function

$f(z) = 1$, if $z \geq 0$

$f(z) = 0$, if $z < 0$

3. Perceptron Weight Update Rule

$w(new) = w(old) + \eta \times (y - \hat{y}) \times x$

4. Bias Update

$b(new) = b(old) + \eta \times (y - \hat{y})$

5. Sigmoid Activation

$\sigma(z) = 1 / (1 + e^{-z})$

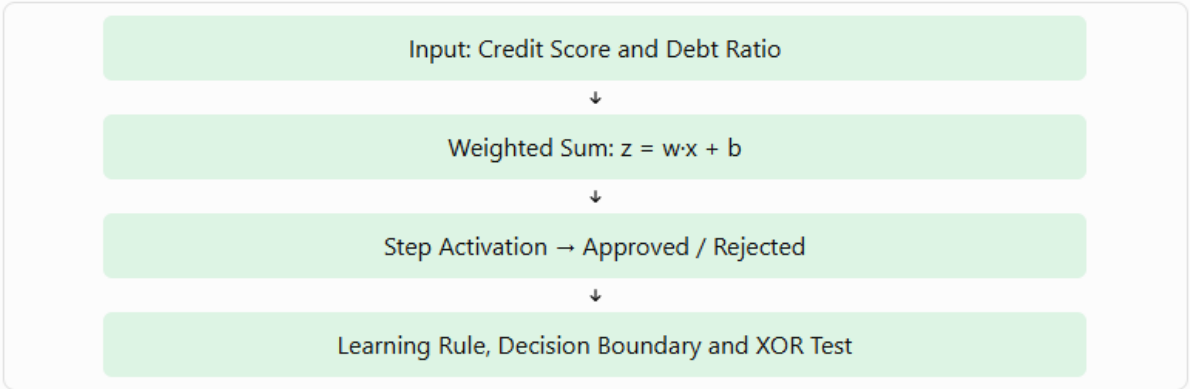
6. ReLU Activation

$ReLU(z) = \max(0, z)$

7. Perceptron Prediction

$\hat{y} = f(WX + b)$

Architecture diagram



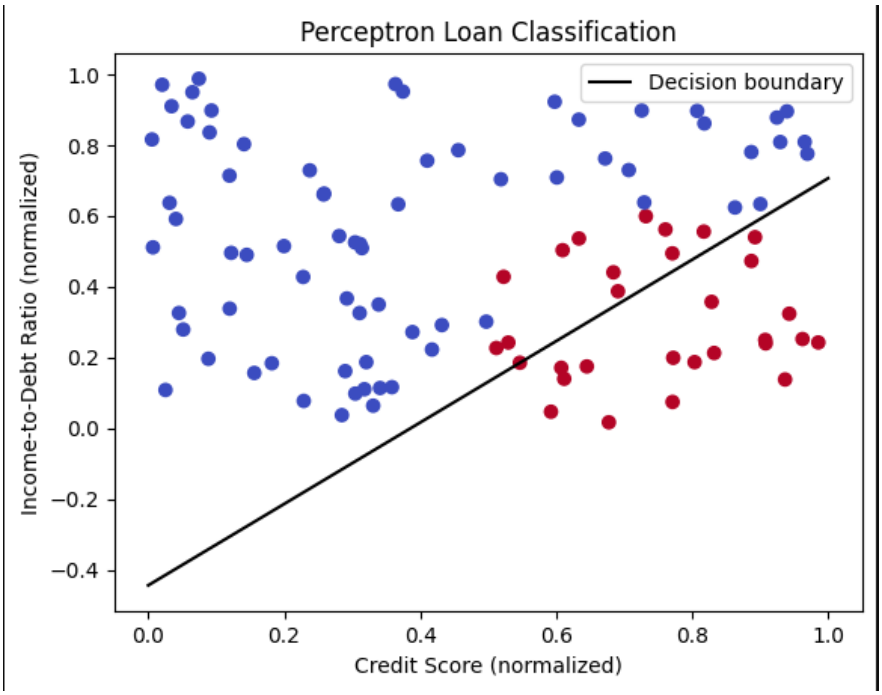
Code

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
X = np.random.rand(100, 2)
y = ((X[:, 0] > 0.5) & (X[:, 1] < 0.6)).astype(int)
def step(z):
    return (z >= 0).astype(int)
def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))
def relu(z):
    return np.maximum(0, z)
values = np.array([-2, -1, 0, 1, 2])
print("Input:", values)
print("Step:", step(values))
print("Sigmoid:", np.round(sigmoid(values), 3))
print("ReLU:", relu(values))
w = np.zeros(2)
b = 0.0
lr = 0.1
errors_per_epoch = []
for epoch in range(50):
    errors = 0
    for xi, target in zip(X, y):
        prediction = step(np.dot(w, xi) + b)
        error = target - prediction
        w += lr * error * xi
        b += lr * error
        errors += int(error != 0)
    errors_per_epoch.append(errors)
    print(f"Epoch {epoch+1}: weight={w}, bias={b:.3f}, "
```

```
        f"errors={errors}")
plt.scatter(X[:, 0], X[:, 1], c=y, cmap="coolwarm")
xx = np.linspace(0, 1, 100)
if abs(w[1]) > 1e-10:
    yy = -(w[0] * xx + b) / w[1]
    plt.plot(xx, yy, "k-", label="Decision boundary")
plt.xlabel("Credit Score (normalized)")
plt.ylabel("Income-to-Debt Ratio (normalized)")
plt.title("Perceptron Loan Classification")
plt.legend()
plt.show()
XOR_X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
XOR_y = np.array([0, 1, 1, 0])
w = np.zeros(2)
b = 0.0
lr = 0.1
for epoch in range(20):
    errors = 0
    for xi, target in zip(XOR_X, XOR_y):
        pred = step(np.dot(w, xi) + b)
        error = target - pred
        w += lr * error * xi
        b += lr * error
        errors += int(error != 0)
    print(f"XOR Epoch {epoch+1}: errors={errors}")
print("Final XOR predictions:", step(XOR_X @ w + b))
print("Actual XOR labels:", XOR_y)
```

output

```
Input: [-2 -1  0  1  2]
Step: [0 0 1 1 1]
Sigmoid: [0.119 0.269 0.5   0.731 0.881]
ReLU: [0 0 0 1 2]
Epoch 1: weight=[ 0.2336223  -0.21601419], bias=-0.100, errors=27
```




```
XOR Epoch 1: errors=3

Final XOR predictions: [1 1 0 0]
Actual XOR labels: [0 1 1 0]
```

Experiment 8: Two-Layer Neural Network with Backpropagation

Aim

To implement a two-layer neural network from scratch using NumPy, perform forward propagation and backpropagation, train on XOR data, visualize the nonlinear decision boundary, and verify gradients numerically.

Formula

1. Forward Propagation – Hidden Layer

$Z_1 = W_1X + b_1$

$A_1 = \sigma(Z_1)$

2. Forward Propagation – Output Layer

$Z_2 = W_2A_1 + b_2$

$A_2 = \sigma(Z_2)$

3. Sigmoid Activation

$\sigma(z) = 1 / (1 + e^{-z})$

4. Output Layer Error

$\delta_2 = A_2 - Y$

5. Hidden Layer Error

$\delta_1 = (W_2^T \times \delta_2) \times \sigma'(Z_1)$

6. Sigmoid Derivative

$\sigma'(z) = \sigma(z) \times (1 - \sigma(z))$

7. Weight Update

$W(new) = W(old) - \eta \times \partial L/\partial W$

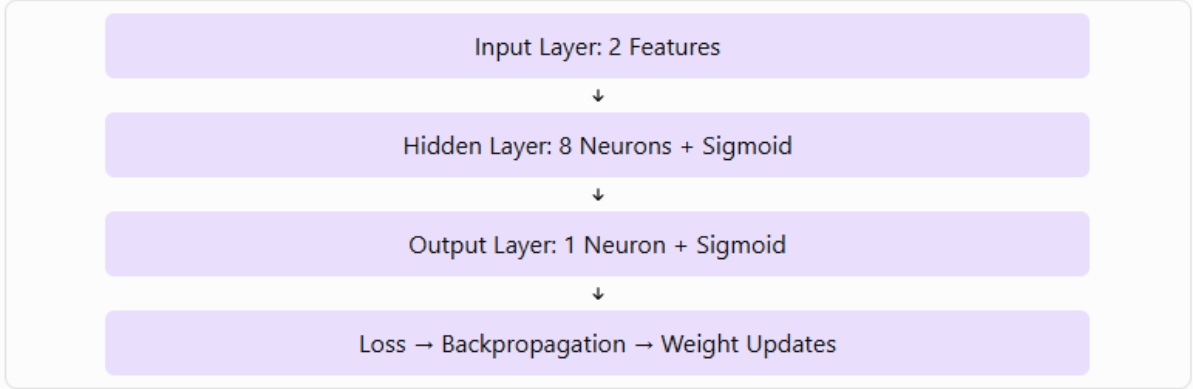
8. Bias Update

$b(new) = b(old) - \eta \times \partial L/\partial b$

9. Binary Cross-Entropy Loss

$L = -[y \log(p) + (1 - y) \log(1 - p)]$

Architecture diagram



Code

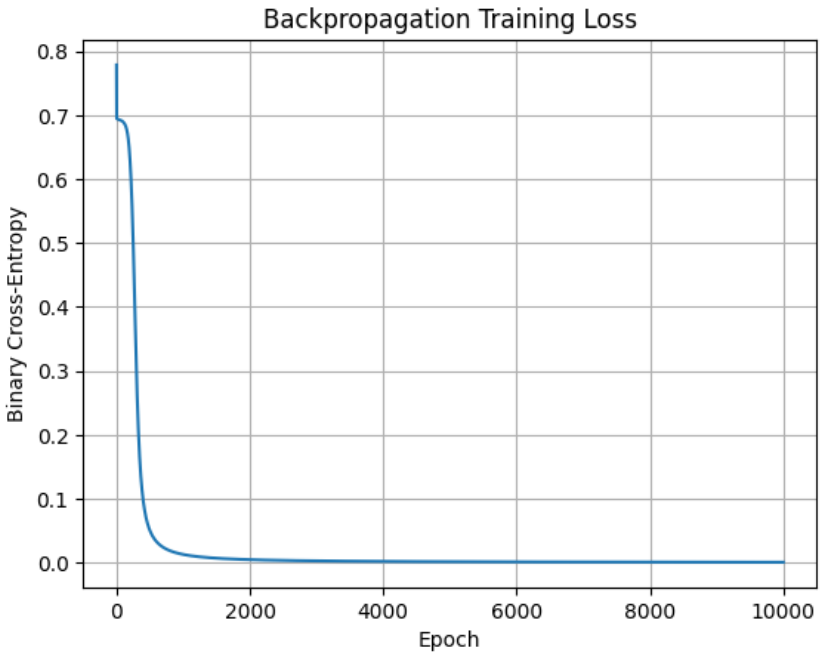
```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([[0], [1], [1], [0]], dtype=float)
def sigmoid(z):
    return 1 / (1 + np.exp(-np.clip(z, -500, 500)))
W1 = np.random.randn(2, 8) * 0.5
b1 = np.zeros((1, 8))
W2 = np.random.randn(8, 1) * 0.5
b2 = np.zeros((1, 1))
lr = 1.0
losses = []
for epoch in range(10000):
    z1 = X @ W1 + b1
    a1 = sigmoid(z1)
    z2 = a1 @ W2 + b2
    a2 = sigmoid(z2)
    eps = 1e-9
    loss = -np.mean(
        y * np.log(a2 + eps)
        + (1-y) * np.log(1-a2 + eps)
    )
    losses.append(loss)
    dz2 = (a2 - y) / len(X)
    dW2 = a1.T @ dz2
    db2 = np.sum(dz2, axis=0, keepdims=True)
    dz1 = (dz2 @ W2.T) * a1 * (1-a1)
    dW1 = X.T @ dz1
    db1 = np.sum(dz1, axis=0, keepdims=True)
    W1 -= lr * dW1
    b1 -= lr * db1
    W2 -= lr * dW2
```

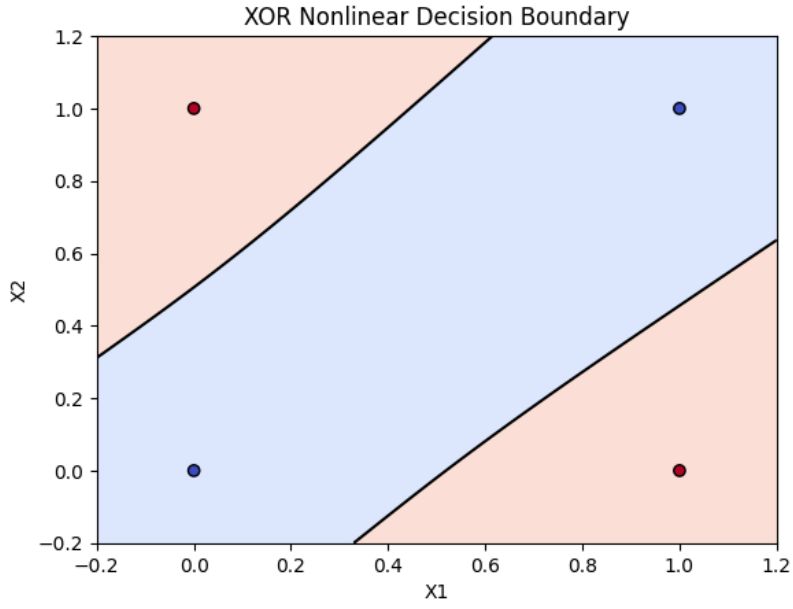
```
    b2 -= lr * db2
predictions = sigmoid(sigmoid(X @ W1 + b1) @ W2 + b2)
print("Predicted probabilities:\n", predictions.round(3))
print("Predicted classes:\n", (predictions >= 0.5).astype(int))
print("Actual labels:\n", y.astype(int))
plt.plot(losses)
plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy")
plt.title("Backpropagation Training Loss")
plt.grid()
plt.show()
grid_x, grid_y = np.meshgrid(
    np.linspace(-0.2, 1.2, 150),
    np.linspace(-0.2, 1.2, 150)
)
grid = np.c_[grid_x.ravel(), grid_y.ravel()]
grid_pred = sigmoid(sigmoid(grid @ W1 + b1) @ W2 + b2)
grid_pred = grid_pred.reshape(grid_x.shape)

plt.contourf(grid_x, grid_y, grid_pred, levels=[0, 0.5, 1],
             alpha=0.3, cmap="coolwarm")
plt.contour(grid_x, grid_y, grid_pred, levels=[0.5], colors="black")
plt.scatter(X[:, 0], X[:, 1], c=y.ravel(),
            cmap="coolwarm", edgecolors="black")
plt.title("XOR Nonlinear Decision Boundary")
plt.xlabel("X1")
plt.ylabel("X2")
plt.show()
def calculate_loss(w2):
    output = sigmoid(sigmoid(X @ W1 + b1) @ w2 + b2)
    return -np.mean(
        y*np.log(output+eps) + (1-y)*np.log(1-output+eps)
    )
analytic = dW2[0, 0]
original = W2[0, 0]
delta = 1e-5
W2[0, 0] = original + delta
loss_plus = calculate_loss(W2)
W2[0, 0] = original - delta
loss_minus = calculate_loss(W2)
W2[0, 0] = original
numeric = (loss_plus - loss_minus) / (2*delta)
print("Analytical gradient:", analytic)
print("Numerical gradient:", numeric)
```

output

```
Predicted probabilities:
 [[0.1]
 [0.999]
 [0.999]
 [0.001]]
Predicted classes:
 [[0]
 [1]
 [1]
 [0]]
Actual labels:
 [[0]
 [1]
 [1]
 [0]]
```





```
Analytical gradient: -2.0035556468093956e-05
Numerical gradient: -2.0033471138502373e-05
```

Experiment 9: Multilayer Perceptron (MLP) Implementation

Aim

To preprocess customer-churn data, encode categorical variables, scale numerical features, build and train an MLP classifier using Keras, evaluate classification performance, and examine the effect of hidden-layer width.

Formula

1. Weighted Sum

$Z = WX + b$

2. ReLU Activation

$ReLU(z) = max(0, z)$

3. Sigmoid Activation

$\sigma(z) = 1 / (1 + e^{-z})$

4. Binary Cross-Entropy Loss

$L = -[y \log(p) + (1 - y) \log(1 - p)]$

5. Accuracy

$Accuracy = (TP + TN) / (TP + TN + FP + FN) \times 100$

6. Precision

$Precision = TP / (TP + FP)$

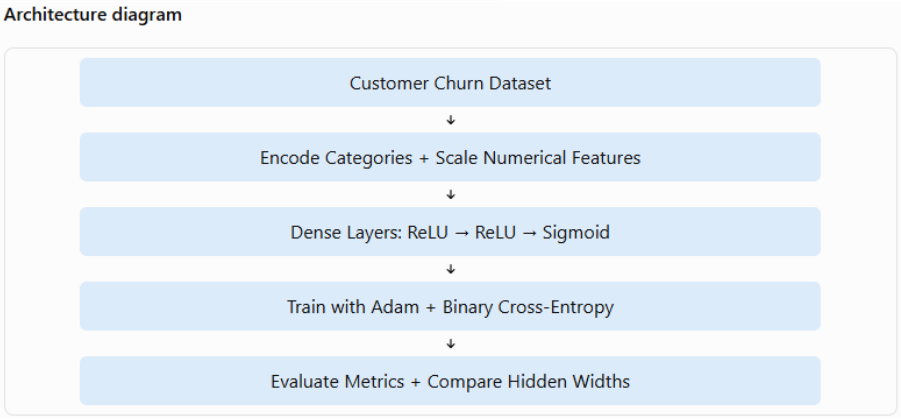
7. Recall

$Recall = TP / (TP + FN)$

8. F1-Score

$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$

Architecture diagram



Code

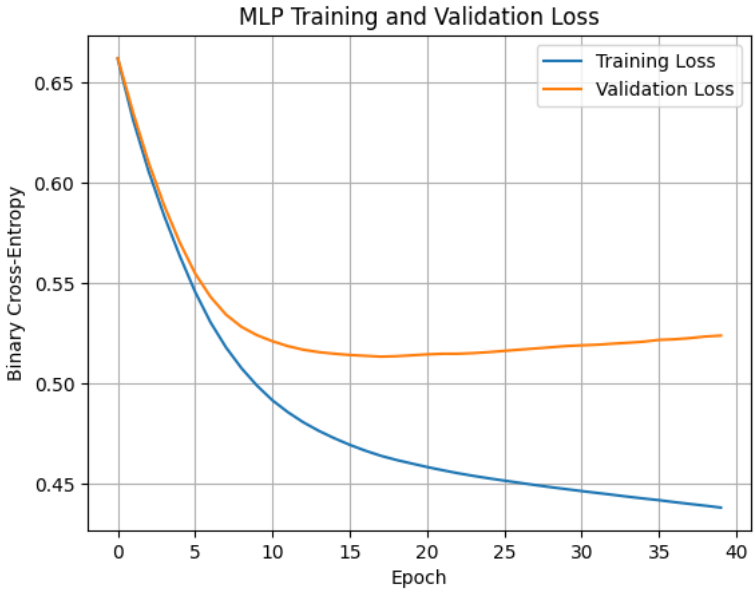
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, ConfusionMatrixDisplay
)
np.random.seed(42)
tf.random.set_seed(42)
n = 500
df = pd.DataFrame({
    "tenure": np.random.randint(1, 73, n),
    "monthly_charges": np.random.uniform(20, 120, n),
    "contract": np.random.choice(
        ["Monthly", "Yearly", "TwoYear"], n
    ),
    "complaints": np.random.poisson(1, n),
    "data_usage": np.random.uniform(1, 100, n)
})
risk = (
    -0.04 * df["tenure"]
    + 0.025 * df["monthly_charges"]
    + 0.5 * df["complaints"]
    + (df["contract"] == "Monthly") * 1.2
    - 2
)
```

```
prob = 1 / (1 + np.exp(-risk))
df["Churn"] = (np.random.rand(n) < prob).astype(int)
X = df.drop(columns="Churn")
y = df["Churn"].to_numpy()
numeric = [
    "tenure", "monthly_charges", "complaints", "data_usage"
]
categorical = ["contract"]
preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numeric),
    ("cat", OneHotEncoder(handle_unknown="ignore",
                          sparse_output=False), categorical)
])
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
X_train = preprocessor.fit_transform(X_train).astype("float32")
X_test = preprocessor.transform(X_test).astype("float32")
def build_model(width=32):
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(X_train.shape[1],)),
        tf.keras.layers.Dense(width, activation="relu"),
        tf.keras.layers.Dense(width // 2, activation="relu"),
        tf.keras.layers.Dense(1, activation="sigmoid")
    ])
    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    return model
```

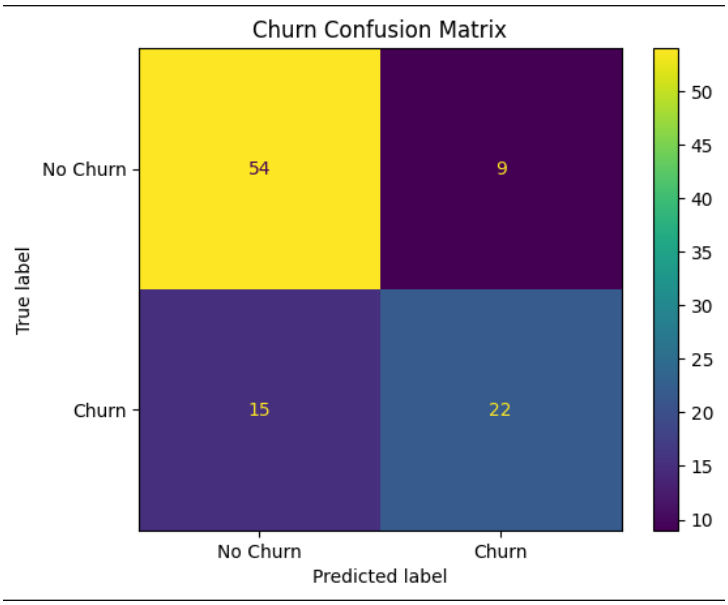
```
model = build_model(32)
history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=40, batch_size=32, verbose=0
)
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy")
plt.title("MLP Training and Validation Loss")
plt.legend()
plt.grid()
plt.show()
pred = (model.predict(X_test, verbose=0).ravel() >= 0.5).astype(int)
print("Accuracy:", round(accuracy_score(y_test, pred), 3))
print("Precision:", round(precision_score(
    y_test, pred, zero_division=0), 3))
print("Recall:", round(recall_score(
    y_test, pred, zero_division=0), 3))
print("F1 score:", round(f1_score(
    y_test, pred, zero_division=0), 3))
ConfusionMatrixDisplay.from_predictions(
    y_test, pred, display_labels=["No Churn", "Churn"]
)
plt.title("Churn Confusion Matrix")
plt.show()
for width in [8, 32, 64]:
    tf.keras.backend.clear_session()
    tf.random.set_seed(42)
    m = build_model(width)
```

```
    m.fit(X_train, y_train, epochs=30,
          batch_size=32, verbose=0)
    p = (m.predict(X_test, verbose=0).ravel() >= 0.5).astype(int)
    print(f"Width {width}: Accuracy = "
          f"{accuracy_score(y_test, p):.3f}, "
          f"F1 = {f1_score(y_test, p, zero_division=0):.3f}")
```

output




```
Accuracy: 0.76
Precision: 0.71
Recall: 0.595
F1 score: 0.647
```



```
Width 8: Accuracy = 0.750, F1 = 0.627
```

Experiment 10: L2, Dropout and Early Stopping Comparison

Aim

To demonstrate overfitting in a large baseline MLP and compare L2 regularization, dropout and early stopping using test accuracy and the train–test accuracy gap.

Formula

1. L2 Regularisation

$$\text{L2 Loss} = \text{Original Loss} + \lambda \times \Sigma w^2$$

2. Dropout

$$\text{Output} = \text{Input} \times \text{Mask} / (1 - p)$$

Where Mask is 0 or 1 during inverted dropout training.

3. Early Stopping

Stop training when validation loss does not improve for a specified number of epochs (patience).

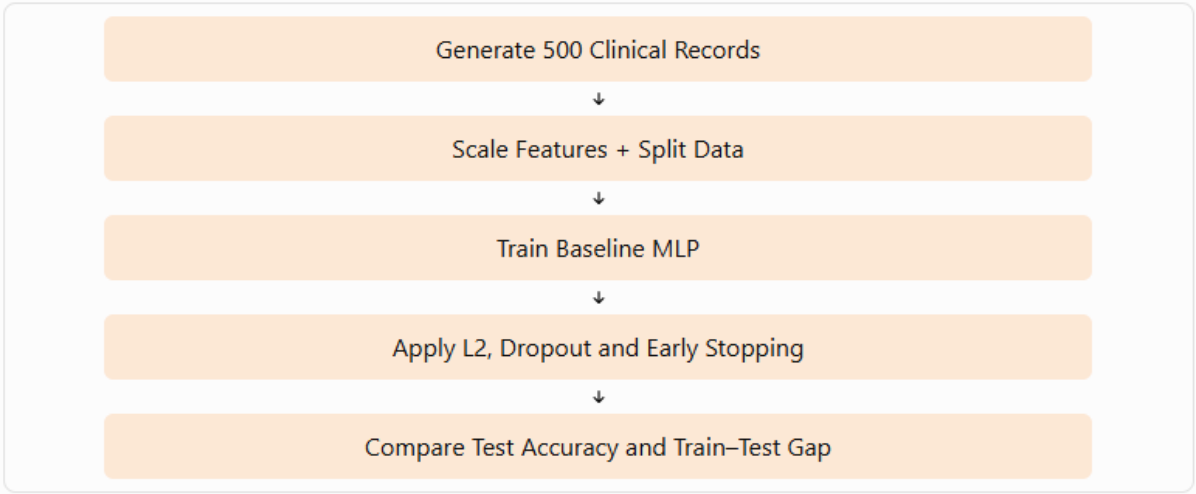
4. Train–Test Gap

$$\text{Train–Test Gap} = \text{Training Accuracy} - \text{Testing Accuracy}$$

5. Mean Squared Error

$$MSE = \Sigma(y_i - \hat{y}_i)^2 / N$$

Architecture diagram



Code

```
import numpy as np

import pandas as pd
```

```
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
np.random.seed(42)
tf.random.set_seed(42)
X, y = make_classification(
    n_samples=500,
    n_features=8,
    n_informative=5,
    n_redundant=2,
    weights=[0.6, 0.4],
    flip_y=0.03,
    random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
def build_model(l2=0, dropout=0):
    regularizer = tf.keras.regularizers.l2(l2) if l2 else None
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(8,)),
        tf.keras.layers.Dense(
            128, activation="relu", kernel_regularizer=regularizer
        ),
        tf.keras.layers.Dense(
```

```
            128, activation="relu", kernel_regularizer=regularizer
        ),
        tf.keras.layers.Dropout(dropout),
        tf.keras.layers.Dense(
            64, activation="relu", kernel_regularizer=regularizer
        ),
        tf.keras.layers.Dense(1, activation="sigmoid")
    ])
    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    return model
results = []
histories = {}
def evaluate_model(name, l2=0, dropout=0, early_stop=False):
    tf.keras.backend.clear_session()
    tf.random.set_seed(42)
    model = build_model(l2, dropout)
    callbacks = []
    if early_stop:
        callbacks.append(tf.keras.callbacks.EarlyStopping(
            monitor="val_loss",
            patience=10,
            restore_best_weights=True
        ))

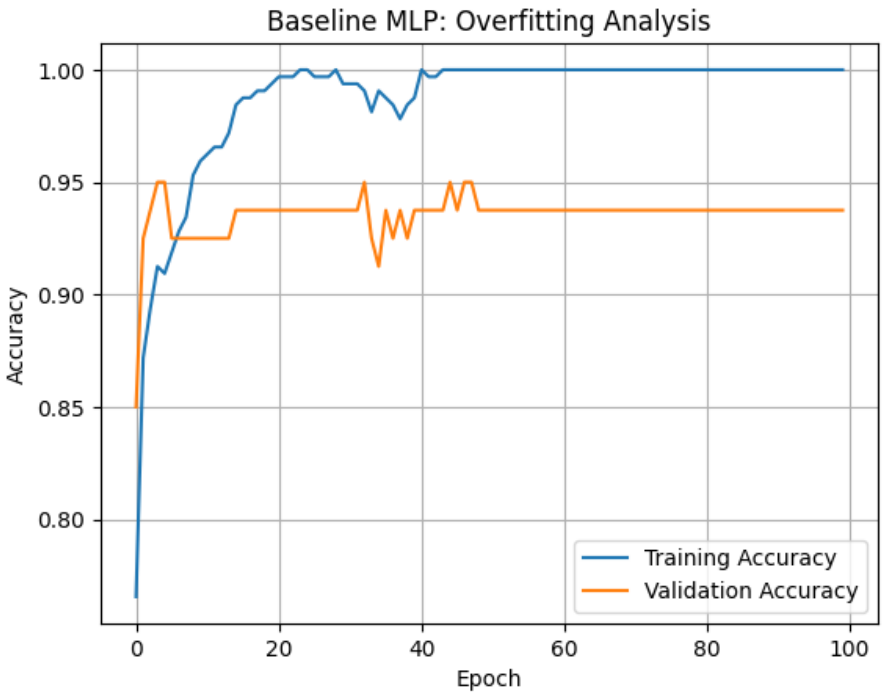
    history = model.fit(
        X_train, y_train,
        validation_split=0.2,
```

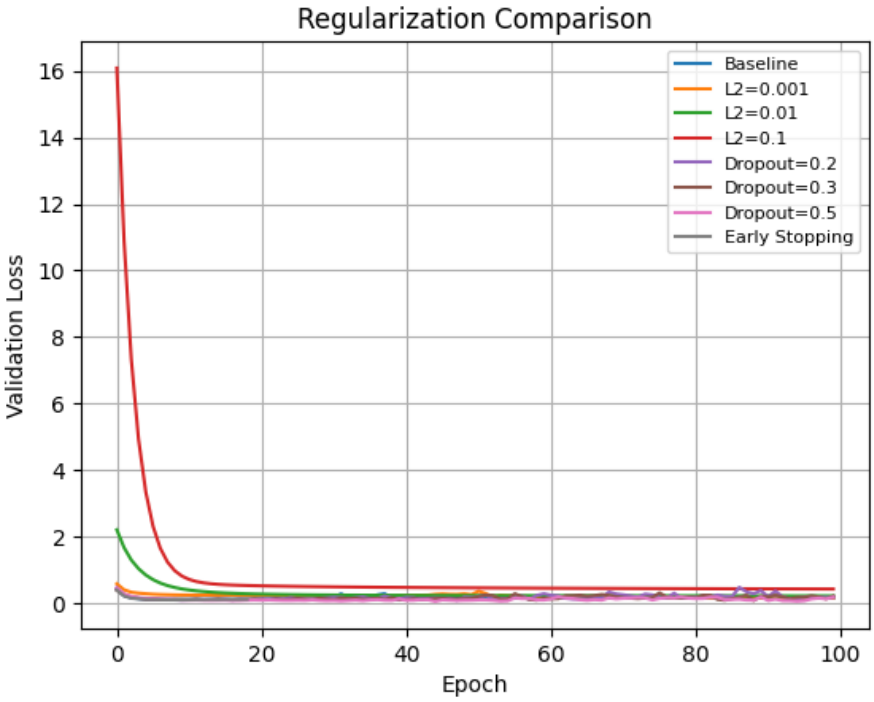
```
        epochs=100,
        batch_size=16,
        callbacks=callbacks,
        verbose=0
    )
    train_pred = (
        model.predict(X_train, verbose=0).ravel() >= 0.5
    ).astype(int)
    test_pred = (
        model.predict(X_test, verbose=0).ravel() >= 0.5
    ).astype(int)
    train_acc = accuracy_score(y_train, train_pred)
    test_acc = accuracy_score(y_test, test_pred)
    results.append({
        "Model": name,
        "Train Accuracy": round(train_acc, 3),
        "Test Accuracy": round(test_acc, 3),
        "Train-Test Gap": round(train_acc - test_acc, 3)
    })
    histories[name] = history.history
evaluate_model("Baseline")
for lam in [0.001, 0.01, 0.1]:
    evaluate_model(f"L2={lam}", l2=lam)
for rate in [0.2, 0.3, 0.5]:
    evaluate_model(f"Dropout={rate}", dropout=rate)
evaluate_model("Early Stopping", early_stop=True)
table = pd.DataFrame(results)
print(table.to_string(index=False))
base = histories["Baseline"]
plt.plot(base["accuracy"], label="Training Accuracy")
plt.plot(base["val_accuracy"], label="Validation Accuracy")
```

```
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Baseline MLP: Overfitting Analysis")
plt.legend()
plt.grid()
plt.show()
for name, h in histories.items():
   plt.plot(h["val_loss"], label=name)
plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.title("Regularization Comparison")
plt.legend(fontsize=8)
plt.grid()
plt.show()
```

output

Model	Train Accuracy	Test Accuracy	Train-Test Gap
Baseline	0.988	0.90	0.088
L2=0.001	0.990	0.88	0.110
L2=0.01	0.968	0.83	0.138
L2=0.1	0.868	0.82	0.048
Dropout=0.2	0.988	0.89	0.098
Dropout=0.3	0.990	0.88	0.110
Dropout=0.5	0.985	0.87	0.115
Early Stopping	0.950	0.88	0.070


Baseline MLP: Overfitting Analysis
Accuracy
Epoch
Training Accuracy
Validation Accuracy
1.00
0.95
0.90
0.85
0.80
0
20
40
60
80
100


Regularization Comparison
Validation Loss
Epoch
Baseline
L2=0.001
L2=0.01
L2=0.1
Dropout=0.2
Dropout=0.3
Dropout=0.5
Early Stopping
16
14
12
10
8
6
4
2
0
0
20
40
60
80
100